

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

```
folder = "/content/drive/MyDrive/FINAL-APP!!!! - 3nd"
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import seaborn as sns
```

```
import folium
```

```
from geopy.geocoders import Nominatim
```

```
import panel as pn
```

```
import plotly.express as px
```

```
import plotly.graph_objects as go
```

```
import re
```

```
df2 = pd.read_csv(
    f"{folder}/ghs-2024-person-v1.csv",
    encoding="windows-1252"
)
```

```
df2.head(5)
```

	uqnr	personnr	psu	prov	Four_by_Four	Sex	age	age_grp1	Population	Languages	...	sevdisab
0	160100090000003901	1	16010009000	Western Cape	Yes	Male	81	75+	White	Afrikaans	...	Not disabled
1	160100090000003901	2	16010009000	Western Cape	Yes	Female	78	75+	White	Afrikaans	...	Disabled
2	160100090000007201	1	16010009000	Western Cape	Yes	Male	27	25–29 years	African/Black	Other language not specified	...	Not disabled
3	160100090000013801	1	16010009000	Western Cape	Yes	Male	36	35–39 years	African/Black	Other language not specified	...	Not disabled
4	160100090000013801	2	16010009000	Western Cape	Yes	Female	30	30–34 years	African/Black	Other language not specified	...	Not disabled

```
5 rows × 114 columns
```

```
for col in df2.select_dtypes(include='object').columns:
    df2[col] = df2[col].str.replace(r'---', '-', regex=True)
```

```
columns_to_convert = ['uqnr', 'personnr', 'psu']
```

```
for col in columns_to_convert:
```

```
    if col in df2.columns:
```

```
        df2[col] = df2[col].astype(str)
```

▼ Filter 'hhc_fath_alive' == 'Yes' and hhc_fath_parthh - 'Yes' or 'No'

```
filter_df = df2[
    (df2['hhc_fath_alive'] == 'Yes') &
    (df2['hhc_fath_parthh'].isin(['Yes', 'No']))
]
```

```
]

```

```
# Count the instances of each response in the 'hhc_fath_alive' column
response_counts = filter_df['hhc_fath_parthh'].value_counts()
response_counts

```

count	
hhc_fath_parthh	
No	25453
Yes	11430

dtype: int64

Start coding or [generate](#) with AI.

▼ Data exploration

```
# Assuming df2 is your DataFrame
null = df2.isnull().sum().reset_index()
null.columns = ['Column', 'Missing Values']

```

null

1 to 25 of 114 entries Filter ?

index	Column	Missing Values
0	uqnr	0
1	personnr	0
2	psu	0
3	prov	0
4	Four_by_Four	0
5	Sex	0
6	age	0
7	age_grp1	0
8	Population	0
9	Languages	0
10	hhc_relationship	0
11	hhc_marital	0
12	spouse	0
13	spouse_name	0
14	hhc_fath_alive	0
15	hhc_fath_parthh	0
16	hhc_fath_name	0
17	hhc_moth_alive	0
18	hhc_moth_parthh	0
19	hhc_moth_name	0
20	education	0
21	ecd_chldatt	0
22	ecd_notatt	0
23	ecd_notatt_rsn	0
24	ecd_fees	0

Show 25 per page 1 2 3 4 5

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Next steps: [Generate code with null](#) [New interactive sheet](#)

```
# Step 1: Get column data types and convert them to string
dtype = df2.dtypes.reset_index()
dtype.columns = ['Column', 'Data type']
dtype['Data type'] = dtype['Data type'].astype(str) # <--- Convert to str

dtype

```

1 to 25 of 114 entries

Filter

?

index	Column	Data type
0	uqnr	object
1	personnr	object
2	psu	object
3	prov	object
4	Four_by_Four	object
5	Sex	object
6	age	int64
7	age_grp1	object
8	Population	object
9	Languages	object
10	hhc_relationship	object
11	hhc_marital	object
12	spouse	object
13	spouse_name	object
14	hhc_fath_alive	object
15	hhc_fath_parthh	object
16	hhc_fath_name	object
17	hhc_moth_alive	object
18	hhc_moth_parthh	object
19	hhc_moth_name	object
20	education	object
21	ecd_chldatt	object
22	ecd_notatt	object
23	ecd_notatt_rsn	object
24	ecd_fees	object

Show 25 per page

12345

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Next steps:

Generate code with dtype

New interactive sheet

```
summary = df2.describe()
summary
```

	age	stratum	person_wgt	
count	70440.000000	70440.000000	70440.000000	
mean	30.488742	55483.675681	896.923133	
std	20.900481	26127.161877	453.695778	
min	0.000000	10101.000000	50.000000	
25%	13.000000	30103.000000	607.616305	
50%	28.000000	55201.000000	821.170963	
75%	45.000000	78103.000000	1087.833970	
max	110.000000	90401.000000	5530.993348	

Next steps:

Generate code with summary

New interactive sheet

```
## Check for duplicates considering ALL columns
duplicates = df2.duplicated().sum()
print(f"Duplicated rows for df2: {duplicates}")

#clean_data = clean_data.drop_duplicates()
```

Duplicated rows for df2: 0

```
df2.shape

(70440, 114)
```

Start coding or [generate](#) with AI.

▼ Data transformation (Label encoding - by mapping)

```

df3 = pd.read_csv(
    f"{folder}/ghs-2023-person-v1.csv",
    encoding="windows-1252"
)

for col in df3.select_dtypes(include='object').columns:
    df3[col] = df3[col].str.replace(r'---', '-', regex=True)

columns_to_convert = ['uqnr', 'personnr', 'psu']
for col in columns_to_convert:
    if col in df3.columns:
        df3[col] = df3[col].astype(str)

```

```

def data_cleaning(df2, df3):
    """
    Cleans df2 using mappings from df3 and returns:
    - cleaned DataFrame
    - list of columns where not all values were affected by the mapping
    - dictionary of unmapped values per partially affected column
    """

    # Make a copy to avoid modifying original
    df2 = df2.copy()

    # 1. Remove 'years' from 'age_grp1' safely
    if 'age_grp1' in df2.columns:
        df2['age_grp1'] = df2['age_grp1'].apply(
            lambda x: x.replace("years", "").strip() if isinstance(x, str) else x
        )

    # 2. Replace 'No fees' with 'None' in fee-related columns
    for col in ['ecd_fees']:
        if col in df2.columns:
            df2[col] = df2[col].replace('No fees', 'None')

    exclude = [
        'uqnr', 'personnr', 'psu', 'stratum', 'person_wgt', 'age',
        'edu_mode_tr', 'edu_totfees', 'lab_salc', 'lab_transp',
        'lab_res_name', 'lab_salary', 'lab_sto'
    ]

    categorical_columns = df2.select_dtypes(include=['object']).columns.tolist()
    clean_df = df2.dropna(subset=categorical_columns).copy()

    partially_affected_columns = []
    unmapped_values = {}

    for col in df3.columns:
        if col in clean_df.columns and col not in exclude:
            original_col = clean_df[col].copy()

            # Create mapping from suffix to full string
            replacements = []
            for _, row in df3.iterrows():
                val = row[col]
                if isinstance(val, str) and ' ' in val:
                    _, suffix = val.split(' ', 1)
                    replacements.append((suffix.strip(), val.strip()))

            mapping_dict = {suffix: full_val for suffix, full_val in replacements}
            unmapped_in_col = set()

            def replace_value(cell):
                cell_str = str(cell).strip()
                if cell_str in mapping_dict:
                    return mapping_dict[cell_str]
                else:
                    unmapped_in_col.add(cell_str)
                    return cell

            clean_df[col] = clean_df[col].apply(replace_value)

            changed = original_col != clean_df[col]
            if not changed.all():
                partially_affected_columns.append(col)

```

```

unmapped_values[col] = list(unmapped_in_col)

clean_df = clean_df.drop(columns=['spouse', 'spouse_name', 'metro_code'], errors='ignore')

return clean_df, partially_affected_columns, unmapped_values

# Example usage
clean_df, not_fully_affected, unmapped_vals = data_cleaning(df2, df3)

```

```

def apply_code_mappings(df):
    def normalize_label(x):
        if isinstance(x, str):
            # Unify en-dash and em-dash to hyphen
            x = re.sub(r'--', '-', x)
            # Replace non-breaking spaces with normal spaces
            x = x.replace('\u00A0', ' ')
            # Remove spaces around dashes for consistency
            x = re.sub(r'\s*-\s*', '-', x)
            # Collapse multiple spaces to single
            x = ' '.join(x.split())
        return x

    df_copy = df.copy()

    mappings = {
        'education': {
            21: "HIGHER/NATIONAL/ADVANCE CERTIFICATE WITH GRADE 12/STD 10/OCCUPATIONAL CERTIFICATE - NQF LEVEL 5",
            10: "GRADE 10/STANDARD 8/FORM 3/NCV LEVEL 2/OCCUPATIONAL CERTIFICATE - NQF LEV EL 2",
            11: "GRADE 11/STANDARD 9/FORM 4/NCV LEVEL 3/OCCUPATIONAL CERTIFICATE - NQF LEV EL 3",
            12: "GRADE 12/STANDARD 10/FORM 5/NATIONAL SENIOR CERTIFICATE/MATRIC/ NCV LEVEL 4 /OCCUPATIONAL CERTIFICATE - NQF LE",
            24: "BACHELOR'S DEGREE/OCCUPATIONAL CERTIFICATE - NQF LEVEL 7"
        },
        'ecd_chldatt': {
            2: "Grade R in pre-school that is linked to a primary school"
        },
        'ecd_notatt': {
            5: "Other places than any of the above",
            4: "At someone else's dwelling"
        },
        'edu_edui': {
            4: "Literacy classes (e.g. Kha Ri Gude)"
        },
        'edu_mode_tr': {
            1: "Walking", 2: "Bicycle", 3: "Motorcycle",
            4: "Minibus taxi/sedan taxi/bakkie taxi",
            5: "Minibus/bus provided by institution/government and not paid for",
            6: "Train", 7: "Bus (public transport)",
            8: "Vehicle hired by a group of parents",
            9: "Own car or other private vehicle",
            10: "Other means of transport",
            88: "Not applicable"
        },
        'edu_time': {
            6: "DO NOT KNOW"
        },
        'edu_totfees': {
            0: "No Fees", 1: "R1-R100", 2: "R101-R200", 3: "R201-R300",
            4: "R301-R500", 5: "R501-R1 000", 6: "R1 001-R2 000", 7: "R2 001-R3 000",
            8: "R3 001-R4 000", 9: "R4 001-R8 000", 10: "R8 001-R12 000",
            11: "R12 001-R16 000", 12: "R16 001-R20 000", 13: "R20 001-R40 000",
            14: "R40 001-R80 000", 15: "More than R80 000", 16: "DO NOT KNOW",
            88: "Not applicable"
        },
        'lab_salc': {
            1: "R1-R2 400", 2: "R2 401-R6 000", 3: "R6 001-R12 000",
            4: "R12 001-R18 000", 5: "R18 001-R30 000", 6: "R30 001-R42 000",
            7: "R42 001-R54 000", 8: "R54 001-R72 000", 9: "R72 001-R96 000",
            10: "R96 001-R132 000", 11: "R132 001-R192 000", 12: "R192 001-R360 000",
            13: "R360 001-R450 000", 14: "R450 001-R650 000", 15: "R650 001-R750 000",
            16: "R750 001-R850 000", 17: "R850 001-R1 000 000", 18: "R1 000 001 or more",
            19: "DO NOT KNOW", 20: "REFUSED", 88: "Not applicable"
        },
        'lab_transp': {
            1: "Office is at home", 2: "Walking", 3: "Bicycle", 4: "Motorcycle",
            5: "Minibus taxi/sedan taxi/bakkie taxi", 6: "Bus", 7: "Train",

```

```

        8: "Lift club by a group of people sharing a private vehicle",
        9: "Own car/other private vehicle", 10: "Company car/staff transport",
        11: "Other means of transport", 88: "Not applicable"
    },
    'lab_mins': {
        6: "DO NOT KNOW"
    },
    'geotype': {
        1: "Urban", 2: "Traditional", 3: "Farms"
    },
    'lab_salary': {
        888888888: 'Not applicable', 999999999: 'Unspecified'
    },
    'ecd_care': {
        1: "Biological / adoptive / foster father",
        2: "Biological / adoptive / foster mother",
        3: "Grandfather",
        4: "Grandmother",
        5: "Sister",
        6: "Brother",
        7: "Cousin",
        8: "Aunt",
        9: "Uncle",
        10: "Other relative",
        11: "Paid carer/nanny",
        12: "Someone else",
        13: "NO ONE",
        88: "Not applicable"
    },
    'ecd_birth_cert': {
        1: "Yes",
        2: "No",
        3: "Do not know",
        8: "Not applicable"
    }
}

for col, mapping in mappings.items():
    if col in df_copy.columns:
        # Normalize mapping keys
        label_to_prefixed = {
            normalize_label(v): f"{k}. {v}" for k, v in mapping.items()
        }
        # Normalize data values
        df_copy[col] = df_copy[col].apply(normalize_label)
        df_copy[col] = df_copy[col].replace(label_to_prefixed)

# Special handling for other specific replacements
if 'edu_grde' in df_copy.columns:
    df_copy['edu_grde'] = df_copy['edu_grde'].replace('14', "14. NC(V) Level 3 (N1/NTC 2)")
if 'lab_res_name' in df_copy.columns:
    df_copy['lab_res_name'] = df_copy['lab_res_name'].replace('Unspecified', "99. Unspecified")
if 'lab_sto' in df_copy.columns:
    df_copy['lab_sto'] = df_copy['lab_sto'].replace('Do not Know', "888888888. DO NOT KNOW")

return df_copy

clean_df = apply_code_mappings(clean_df)

clean_df

```


1 to 25 of 20000 entries

Filter



index	uqnr	personnr	psu	prov	Four_by_Four	Sex	age	age_grp1	Population	Languages	hhc_relations
0	160100090000003901	1	16010009000	1. Western Cape	1. Yes	1. Male	81	16. 75+	4. White	1. Afrikaans	1. Head/acting head
1	160100090000003901	2	16010009000	1. Western Cape	1. Yes	2. Female	78	16. 75+	4. White	1. Afrikaans	2. Husband/wife/partner household head
2	160100090000007201	1	16010009000	1. Western Cape	1. Yes	1. Male	27	6. 25-29	1. African/Black	14. Other language not specified	1. Head/acting head
3	160100090000013801	1	16010009000	1. Western Cape	1. Yes	1. Male	36	8. 35-39	1. African/Black	14. Other language not specified	1. Head/acting head
4	160100090000013801	2	16010009000	1. Western Cape	1. Yes	2. Female	30	7. 30-34	1. African/Black	14. Other language not specified	2. Husband/wife/partner household head
5	160100090000013801	3	16010009000	1. Western Cape	1. Yes	2. Female	3	1. 00-04	1. African/Black	14. Other language not specified	3. Son/daughter/stepchild of household head
6	160100330000001101	1	16010033000	1. Western Cape	1. Yes	1. Male	55	12. 55-59	2. Coloured	1. Afrikaans	1. Head/acting head
7	160100330000001101	2	16010033000	1. Western Cape	1. Yes	2. Female	49	10. 45-49	2. Coloured	1. Afrikaans	2. Husband/wife/partner household head
8	160100330000001101	3	16010033000	1. Western Cape	1. Yes	2. Female	33	7. 30-34	2. Coloured	1. Afrikaans	3. Son/daughter/stepchild of household head
9	160100330000001101	4	16010033000	1. Western Cape	1. Yes	1. Male	21	5. 20-24	2. Coloured	1. Afrikaans	3. Son/daughter/stepchild of household head
10	160100330000001101	5	16010033000	1. Western Cape	1. Yes	1. Male	10	3. 10-14	2. Coloured	1. Afrikaans	7. Grandchild/great grandchild of household head

11	160100330000006101	1	16010033000	1. Western Cape	1. Yes	1. Male	73	15. 70-74	2. Coloured	1. Afrikaans	1. Head/acting head
12	160100330000006101	2	16010033000	1. Western Cape	1. Yes	2. Female	67	14. 65-69	4. White	1. Afrikaans	2. Husband/wife/partner household head
13	160100330000008601	1	16010033000	1. Western Cape	1. Yes	1. Male	49	10. 45-49	2. Coloured	1. Afrikaans	1. Head/acting head
14	160100330000008601	2	16010033000	1. Western Cape	1. Yes	1. Male	23	5. 20-24	2. Coloured	1. Afrikaans	3. Son/daughter/stepchild of household head
15	160100330000013601	1	16010033000	1. Western Cape	1. Yes	1. Male	45	10. 45-49	4. White	1. Afrikaans	1. Head/acting head
16	160100330000013601	2	16010033000	1. Western Cape	1. Yes	2. Female	44	9. 40-44	4. White	1. Afrikaans	2. Husband/wife/partner household head
17	160100330000013601	3	16010033000	1. Western Cape	1. Yes	2. Female	66	14. 65-69	4. White	1. Afrikaans	5. Father/mother/stepfather of household head
18	160100330000013601	4	16010033000	1. Western Cape	1. Yes	2. Female	11	3. 10-14	4. White	1. Afrikaans	3. Son/daughter/stepchild of household head
19	160100630000003601	1	16010063000	1. Western Cape	1. Yes	1. Male	43	9. 40-44	2. Coloured	1. Afrikaans	1. Head/acting head
20	160100630000003601	2	16010063000	1. Western Cape	1. Yes	2. Female	36	8. 35-39	2. Coloured	1. Afrikaans	2. Husband/wife/partner household head
21	160100630000003601	3	16010063000	1. Western	1. Yes	1. Male	17	4. 15-19	2. Coloured	1. Afrikaans	3. Son/daughter/stepchild of household head

Start coding or [generate](#) with AI.

▼ Data Transformation (Cleaning data to remain with codes)

22	160100630000007201	1	16010063000	1.	1. Yes	1. Male	39	8. 35-39	2. Coloured	1. Afrikaans	1. Head/acting head
----	--------------------	---	-------------	----	--------	---------	----	----------	-------------	--------------	---------------------

```
def data_cleaning(data):
    data = pd.DataFrame(data)

    def keep_numbers(cell):
        cell = str(cell).strip()
        match = re.match(r'^(\d+)\.', cell)
        if match:
            return int(match.group(1))
        elif cell.isdigit():
            return int(cell)
        else:
            return np.nan          # Explicitly mark as NaN

    for col in data.select_dtypes(include=['object']).columns:
        data[col] = data[col].apply(keep_numbers)

    return data

clean_data = data_cleaning(clean_df)          # Apply your cleaning fuction

for col in clean_data.select_dtypes(include='object').columns:
    clean_data[col] = clean_data[col].str.replace(r'---', '-', regex=True)

columns_to_convert = ['uqnr', 'personnr', 'psu', 'stratum', 'lab_salary', 'person_wgt', 'lab_sto']
for col in columns_to_convert:
    if col in clean_data.columns:
        clean_data[col] = clean_data[col].astype(str)

clean_data          #Show table
```

1 to 25 of 20000 entries

Filter



index	uqnr	personnr	psu	prov	Four_by_Four	Sex	age	age_grp1	Population	Languages	hhc_relationship	hhc_marital
0	160100090000003901	1	16010009000	1	1	1	81	16	4	1	1	1
1	160100090000003901	2	16010009000	1	1	2	78	16	4	1	2	1
2	160100090000007201	1	16010009000	1	1	1	27	6	1	14	1	1
3	1601000900000013801	1	16010009000	1	1	1	36	8	1	14	1	2
4	1601000900000013801	2	16010009000	1	1	2	30	7	1	14	2	2
5	1601000900000013801	3	16010009000	1	1	2	3	1	1	14	3	7
6	160100330000001101	1	16010033000	1	1	1	55	12	2	1	1	1
7	160100330000001101	2	16010033000	1	1	2	49	10	2	1	2	1
8	160100330000001101	3	16010033000	1	1	2	33	7	2	1	3	7
9	160100330000001101	4	16010033000	1	1	1	21	5	2	1	3	7
10	160100330000001101	5	16010033000	1	1	1	10	3	2	1	7	7
11	1601003300000006101	1	16010033000	1	1	1	73	15	2	1	1	1
12	1601003300000006101	2	16010033000	1	1	2	67	14	4	1	2	1
13	1601003300000008601	1	16010033000	1	1	1	49	10	2	1	1	1
14	1601003300000008601	2	16010033000	1	1	1	23	5	2	1	3	7
15	1601003300000013601	1	16010033000	1	1	1	45	10	4	1	1	1
16	1601003300000013601	2	16010033000	1	1	2	44	9	4	1	2	1
17	1601003300000013601	3	16010033000	1	1	2	66	14	4	1	5	5
18	1601003300000013601	4	16010033000	1	1	2	11	3	4	1	3	7
19	1601006300000003601	1	16010063000	1	1	1	43	9	2	1	1	2
20	1601006300000003601	2	16010063000	1	1	2	36	8	2	1	2	2
21	1601006300000003601	3	16010063000	1	1	1	17	4	2	1	3	7
22	1601006300000007201	1	16010063000	1	1	1	39	8	2	1	1	2
23	1601006300000007201	2	16010063000	1	1	2	30	7	2	1	2	2
24	1601006300000007201	3	16010063000	1	1	2	1	1	2	1	3	7

Show 25 per page

1

2

10

100

700

790

800

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Warning: Total number of columns (111) exceeds max_columns (20) limiting to first (20) columns.

Warning: total number of rows (70440) exceeds max_rows (20000). Limiting to first (20000) rows.

Start coding or [generate](#) with AI.

▼ Data Cleaning (filtered data for machine learning)

```
# Filter rows where father is alive and part/not part of household (1 or 2)
num_data = clean_data[
    (clean_data['hhc_fath_alive'] == 1) &
    (clean_data['hhc_fath_parthh'].isin([1, 2]))
].copy() # Use .copy() to avoid SettingWithCopyWarning

# Convert hhc_fath_parthh to binary: 1 -> 1, 2 -> 0
num_data['hhc_fath_parthh'] = num_data['hhc_fath_parthh'].map({1: 1, 2: 0})

# Display the filtered and transformed table

num_data
```

	uqnr	personnr	psu	prov	Four_by_Four	Sex	age	age_grp1	Population	Languages	...	disab	sevd
2	160100090000007201	1	16010009000	1		1	1	27	6	1	14	...	0
3	160100090000013801	1	16010009000	1		1	1	36	8	1	14	...	0
4	160100090000013801	2	16010009000	1		1	2	30	7	1	14	...	0
5	160100090000013801	3	16010009000	1		1	2	3	1	1	14	...	0
9	160100330000001101	4	16010033000	1		1	1	21	5	2	1	...	0
...	...	...	...	...		...	...	...	...	...	...	...	...
70432	987106920000042001	4	98710692000	9		1	1	7	2	1	7	...	0
70434	987106920000047801	2	98710692000	9		1	2	26	6	1	7	...	0
70437	987106920000053601	3	98710692000	9		1	2	8	2	1	7	...	0
70438	987106920000053601	4	98710692000	9		1	2	5	2	1	7	...	0
70439	987106920000053601	5	98710692000	9		1	2	4	1	1	7	...	0

36883 rows × 111 columns

Start coding or [generate](#) with AI.

Dashboard - Findings

```
# Initialize Panel extension
pn.extension('plotly')
```

```
def plot_father_role_by_age_group(df):
    """
    Returns a Panel layout containing a stacked bar chart showing
    father's role in household by age group, using filtered data.
    """

    # Step 2: Ensure columns are strings
    df.loc[:, 'hhc_fath_parthh'] = df['hhc_fath_parthh'].astype(str)
    df.loc[:, 'age_grp1'] = df['age_grp1'].astype(str)

    # Step 3: Create Crosstab and reset index for Plotly
    crosstab = pd.crosstab(
        filter_df['age_grp1'],
        filter_df['hhc_fath_parthh']
    ).reset_index()

    # Step 4: Melt the crosstab to long format
    crosstab_melted = crosstab.melt(
        id_vars='age_grp1',
        var_name='hhc_fath_parthh',
        value_name='Count'
    )

    # Step 5: Plot using Plotly Express
    fig = px.bar(
        crosstab_melted,
        x='age_grp1',
        y='Count',
        color='hhc_fath_parthh',
        title="Distribution of Father Presence at individual level by Age Group<br>(Only where Father is known to be alive)",
        labels={
            'age_grp1': 'Age Group',
            'Count': 'Count',
            'hhc_fath_parthh': "Father Part of HH"
        },
        barmode='stack'
    )

    fig.update_layout(
        xaxis_tickangle=-45,
        legend_title_text='Father Presence',
```

```

        margin=dict(t=80, r=20, l=40, b=80)
    )

    # Return as Panel layout
    plotly_panel = pn.pane.Plotly(fig)
    return pn.Column(plotly_panel)

```

```

def plot_province_vs_father_presence(df):
    """
    Returns a Panel layout containing a normalized stacked bar chart
    of provinces vs father's presence in household,
    excluding 'Not applicable', 'Unspecified', and 'No' values.
    """

    # Step 2: Crosstab (normalized by province)
    crosstab = pd.crosstab(
        df['prov'],
        df['hhc_fath_parthh'],
        normalize='index'
    ).reset_index()

    # Step 3: Melt for Plotly
    df_melted = crosstab.melt(
        id_vars='prov',
        var_name='hhc_fath_parthh',
        value_name='Percentage'
    )

    # Step 4: Plot with Plotly
    fig = px.bar(
        df_melted,
        x='prov',
        y='Percentage',
        color='hhc_fath_parthh',
        barmode='relative',
        title="Distribution of Father Presence at individual level by provinces<br>(Only where father is known to be alive)",
        labels={'prov': 'Province', 'Percentage': 'Proportion'}
    )

    fig.update_layout(
        xaxis_title='Province',
        yaxis_title='Proportion',
        yaxis_tickformat='%.0%',
        legend_title="Father's Presence"
    )

    # Return as Panel layout
    return pn.Column(pn.pane.Plotly(fig))

```

```

def plot_correlation_heatmap(df):
    """
    Returns a Panel layout containing a correlation heatmap
    for all numeric columns in the DataFrame.
    """

    df = pd.DataFrame(filter_df) # Ensure input is a DataFrame

    # Step 1: Keep only numeric columns
    df_numeric = filter_df.select_dtypes(include=['number'])

    # Step 2: Compute correlation matrix
    df_corr = df_numeric.corr()

    # Step 3: Create Plotly heatmap
    fig = px.imshow(
        df_corr,
        text_auto=True,
        aspect="auto",
        color_continuous_scale="RdBu",
        labels=dict(color="Correlation"),
        title="Correlation Heatmap between Numeric Factors"
    )

```

```
# Step 4: Wrap in Panel layout
plotly_panel = pn.pane.Plotly(fig)
return pn.Column(plotly_panel)
```

```
def plot_father_presence_pie(df):
    """
    Returns a Panel layout containing a pie chart showing the distribution
    of father presence (hhc_fath_parthh), excluding 'Not applicable' responses
    and where father is not alive, unknown, or unspecified.
    """

    # Step 2: Count frequencies
    counts = df['hhc_fath_parthh'].value_counts().reset_index()
    counts.columns = ['hhc_fath_parthh', 'Count']

    # Step 3: Plot pie chart
    fig = px.pie(
        counts,
        names='hhc_fath_parthh',
        values='Count',
        title='Distribution of Father Presence at individual level<br>(Only where father is known to be alive)',
        hole=0.3 # Set to 0.0 for full pie chart
    )

    fig.update_traces(textinfo='percent+label')

    # Return as Panel layout
    plotly_panel = pn.pane.Plotly(fig)
    return pn.Column(plotly_panel)
```

```
def plot_father_presence_by_race(df):
    """
    Returns a Panel layout containing a grouped bar chart showing
    father's presence in household across racial groups,
    excluding 'Not applicable' and where father is not alive, unknown, or unspecified.
    """

    # Step 2: Group data
    count_data = df.groupby(['Population', 'hhc_fath_parthh']).size().reset_index(name='Count')

    # Step 3: Plot grouped bar chart
    fig = px.bar(
        count_data,
        x='Population',
        y='Count',
        color='hhc_fath_parthh',
        barmode='group',
        title='Distribution of Father Presence at individual level by<br>Race (Only where father is known to be alive)',
        labels={'Population': 'Race Group', 'hhc_fath_parthh': 'Father Presence'}
    )

    fig.update_layout(
        xaxis_title='Race Group',
        yaxis_title='Count',
        legend_title='Father Presence'
    )

    # Step 4: Return as Panel layout
    plotly_panel = pn.pane.Plotly(fig)
    return pn.Column(plotly_panel)
```

```
def plot_father_presence_by_race(df):
    """
    Returns a Panel layout containing a grouped bar chart showing
    father's presence in household across racial groups,
    excluding 'Not applicable' and where father is not alive, unknown, or unspecified.
```

```

"""

# Step 2: Group data
count_data = df.groupby(['Population', 'hhc_fath_parthh']).size().reset_index(name='Count')

# Step 3: Plot grouped bar chart
fig = px.bar(
    count_data,
    x='Population',
    y='Count',
    color='hhc_fath_parthh',
    barmode='group',
    title='Distribution of Father Presence at individual level by<br>Race (Only where father is known to be alive)',
    labels={'Population': 'Race Group', 'hhc_fath_parthh': 'Father Presence'}
)

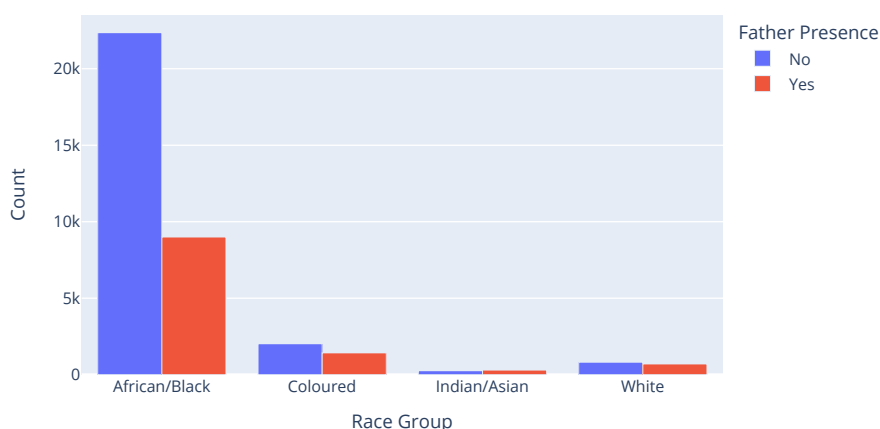
fig.update_layout(
    xaxis_title='Race Group',
    yaxis_title='Count',
    legend_title='Father Presence'
)

# Step 4: Return as Panel layout
plotly_panel = pn.pane.Plotly(fig)
return pn.Column(plotly_panel)

fig = plot_father_presence_by_race(filter_df)
fig

```

Distribution of Father Presence at individual level by
Race (Only where father is known to be alive)



```

def plot_distribution(dataframe, x_col='geotype', color_col='hhc_fath_parthh'):
    """
    Plots a stacked bar chart showing the distribution of hhc_fath_parthh within each geotype using Plotly.

    Parameters:
    - dataframe (pd.DataFrame): The dataset containing the columns.
    - x_col (str): The categorical column to plot on the x-axis (default 'geotype').
    - color_col (str): The column to use for color grouping and stacking (default 'hhc_fath_parthh').
    """
    fig = px.histogram(
        dataframe,
        x=x_col,
        color=color_col,
        barmode='stack',
        title=f'Stacked Distribution of {color_col} by {x_col}',
        labels={x_col: x_col, color_col: color_col},
    )
    fig.update_layout(
        xaxis_title=x_col,
        yaxis_title='Count',
    )

```

```

    bargap=0.1
)

```

```

# Step 4: Return as Panel layout
plotly_panel = pn.pane.Plotly(fig)
return pn.Column(plotly_panel)

```

```

age = plot_father_role_by_age_group(filter_df)
prov = plot_province_vs_father_presence(filter_df)
heatmap = plot_correlation_heatmap(filter_df)
pie_fath = plot_father_presence_pie(filter_df)
race = plot_father_presence_by_race(filter_df)
geontype = plot_distribution(filter_df)

```

```

# Define the button first
tableau_button = pn.widgets.Button(name="Open Tableau Dashboard", button_type="primary")

# Optional: add click behavior
def open_tableau(event):
    import webbrowser
    webbrowser.open("https://public.tableau.com/views/Book2_17141594199830/Dashboard2?:language=en-US&:sid=&:redirect=auth&:dis

tableau_button.on_click(open_tableau)

# Now include it in your visualizations page
def visualizations_page():
    return pn.Column(
        pn.Row(heatmap.clone(width=500, height=400),
            age.clone(width=600, height=400),
            sizing_mode="stretch_width"),
        pn.Row(prov.clone(width=600, height=400),
            race.clone(width=500, height=400),
            sizing_mode="stretch_width"),
        #pn.Row(pie_fath.clone(width=1100, height=400),
            #sizing_mode="stretch_width"),
        pn.Row(pie_fath.clone(width=500, height=400),
            geontype.clone(width=600, height=400),
            sizing_mode="stretch_width"),
        pn.Row(tableau_button, align='center'),
        sizing_mode="stretch_width"
    )

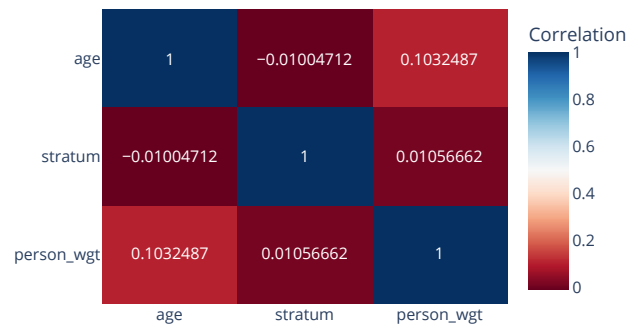
```

```

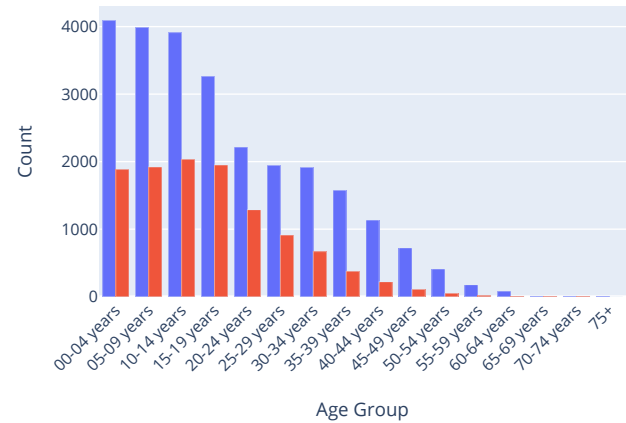
# pip install jupyter_bokeh -
dashboard = visualizations_page()
dashboard.servable()

```

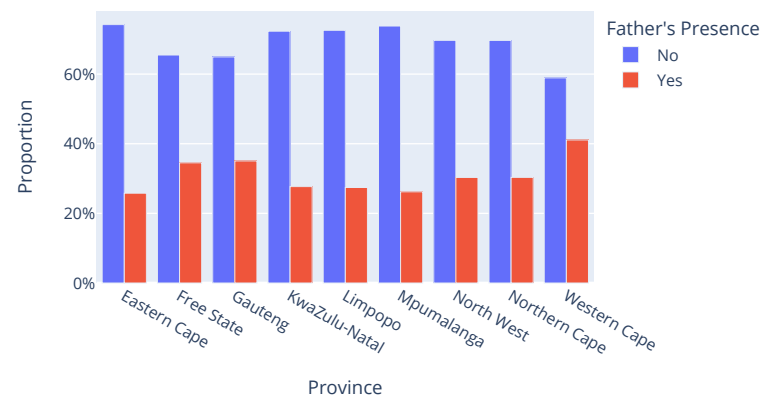
Correlation Heatmap between Numeric Factors



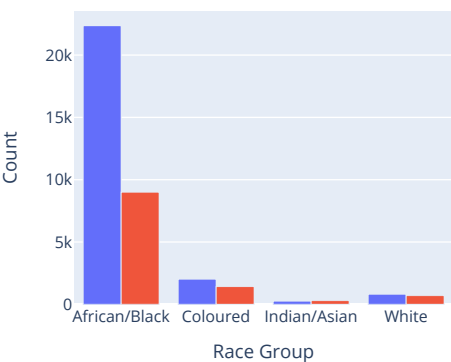
Distribution of Father Presence at individual level by Age (Only where Father is known to be alive)



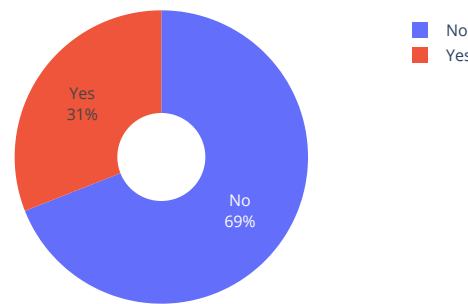
Distribution of Father Presence at individual level by provinces (Only where father is known to be alive)



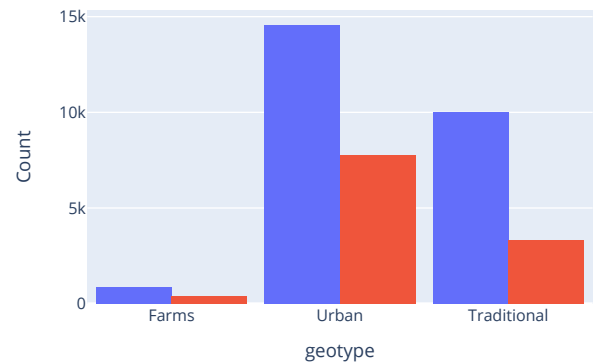
Distribution of Father Presence at individual Race (Only where father is known to be alive)



Distribution of Father Presence at individual level (Only where father is known to be alive)



Stacked Distribution of hhc_fath_parthh by geotype



Open Tableau Dashboard

Start coding or [generate](#) with AI.

- Machine learning
- Connect Python to R